

SYSTEM DESIGN

Comprehensive Study Material

Last Updated: June 30, 2026

TABLE OF CONTENTS

1. Scalability
2. Database Design
3. Caching
4. Message Queues
5. APIs and Microservices
6. Distributed Systems
7. Monitoring and Logging
8. Security
9. Design Patterns
10. Capacity Planning

1. SCALABILITY

Horizontal Scaling: Add more machines to handle increased load. Scale-out approach. Enables handling massive traffic. Requires load balancing and distributed architecture.

Vertical Scaling: Increase resources (CPU, RAM) on single machine. Scale-up approach. Limited by hardware constraints. Simpler but reaches limits quickly.

Load Balancing: Distributes traffic across multiple servers. Round-robin, least connections, weighted algorithms. Prevents single server overload. Enables high availability.

Stateless Design: Services don't store session data. Enables easy horizontal scaling. Session data stored in database or distributed cache. Supports failover.

2. DATABASE DESIGN

SQL vs NoSQL: SQL (relational) offers ACID guarantees, structured schemas. NoSQL (document, key-value, graph) offers flexibility, horizontal scalability. Choose based on consistency vs availability needs.

Normalization: Reduces data redundancy through normal forms (1NF, 2NF, 3NF, BCNF). Improves consistency but can hurt performance due to joins.

Indexing: Creates data structures (B-trees, hash indexes) for faster lookups. Trade-off: faster reads, slower writes, additional storage. Careful index selection critical.

Partitioning/Sharding: Divides data across multiple databases. Range-based, hash-based, directory-based strategies. Enables massive scale but complicates queries.

Replication: Copies data across multiple servers. Master-slave, master-master, peer-to-peer approaches. Improves availability and read performance. Consistency challenges.

3. CACHING

In-Memory Caches: Redis, Memcached store frequently accessed data in RAM. Dramatically faster than database queries. Reduces database load. Critical for high-performance systems.

Cache Invalidation: Two hard problems in computer science. Strategies: TTL (time-based), LRU (eviction), write-through, write-behind. Choose based on consistency requirements.

Cache-Aside Pattern: Application checks cache first. Miss triggers database query and cache update. Simple but clients handle logic.

Write-Through vs Write-Behind: Write-through writes to cache and database synchronously. Write-behind updates cache immediately, database asynchronously. Behind is faster but riskier.

CDN (Content Delivery Network): Geographic distribution of content. Servers near users reduce latency. Caches static assets globally. Essential for global applications.

4. MESSAGE QUEUES

Asynchronous Processing: Decouples producers from consumers. Producers send messages to queue, consumers process at their pace. Improves system resilience.

Event-Driven Architecture: Components communicate via events. Enables loose coupling and independent scaling. Used in microservices and real-time systems.

Kafka: High-throughput distributed streaming platform. Persists messages. Enables event sourcing and stream processing. Partition-based scalability.

RabbitMQ: Message broker supporting multiple protocols. Flexible routing through exchanges and queues. Reliable delivery with acknowledgments.

At-Least-Once Delivery: Messages guaranteed delivery but may duplicate. Consumer must handle idempotently. Trade-off between reliability and performance.

5. APIs AND MICROSERVICES

RESTful Design: Resources identified by URIs. HTTP methods define operations. Stateless, cacheable, uniform interface. Industry standard for web APIs.

gRPC: High-performance RPC framework using HTTP/2. Protocol buffers for serialization. Enables bidirectional streaming. Better than REST for internal services.

Microservices: Decompose monolith into independent services. Each service handles specific domain. Enables independent deployment and scaling. Operational complexity.

API Gateway: Single entry point for client requests. Routes to appropriate services. Handles authentication, rate limiting, request transformation. Simplifies client integration.

Service Discovery: Clients locate services dynamically. Consul, Eureka, Kubernetes DNS. Handles service registration and health checks. Essential for microservices.

6. DISTRIBUTED SYSTEMS

CAP Theorem: Choose two of three: consistency (all nodes see same data), availability (system always operational), partition tolerance (survives network partitions). Trade-offs inherent.

ACID vs BASE: ACID (Atomicity, Consistency, Isolation, Durability) for traditional databases. BASE (Basically Available, Soft state, Eventually consistent) for distributed systems.

Consensus Algorithms: Raft and Paxos ensure distributed agreement. Quorum-based voting. Handle failures. Enable consistent distributed systems.

Eventual Consistency: Relaxes immediate consistency. Replicas converge over time. Enables high availability and partition tolerance. Suitable for many applications.

Distributed Tracing: Follows requests across services. Tools: Jaeger, Zipkin. Identify bottlenecks and latency issues. Critical for debugging microservices.

7. MONITORING AND LOGGING

Observability: Understand system behavior through metrics, logs, traces. Goes beyond monitoring. Enables effective debugging. Three pillars: metrics, logs, traces.

Metrics: Quantitative measurements (CPU, memory, request latency, error rates). Time-series databases store metrics. Prometheus, Grafana visualization.

Logging: Structured logging with key-value pairs. ELK stack (Elasticsearch, Logstash, Kibana) for analysis. Centralized logging critical for distributed systems.

Alerting: Proactive notification on anomalies. Rules based on metrics and thresholds. Enables quick response. Avoid alert fatigue.

SLIs, SLOs, SLAs: Service Level Indicators measure performance. Service Level Objectives are targets. Service Level Agreements are guarantees to users.

8. SECURITY

Authentication: Verify user identity. OAuth 2.0 for delegation, SAML for enterprise SSO. JWT tokens stateless and scalable.

Authorization: Control what authenticated users can do. Role-Based Access Control (RBAC), Attribute-Based (ABAC). Enforce principle of least privilege.

Encryption: In-transit (TLS/HTTPS) and at-rest (AES). Protects confidentiality. Key management critical. Hardware Security Modules for sensitive keys.

DDoS Protection: Rate limiting, traffic filtering, anycast networks. Cloud providers offer scrubbing centers. WAF (Web Application Firewall) blocks common attacks.

Data Privacy: GDPR, CCPA compliance. Data minimization, consent management. Privacy-by-design principles. Regular security audits.

9. DESIGN PATTERNS

MVC (Model-View-Controller): Separates concerns. Model handles data, view displays, controller processes requests. Enables testing and maintenance.

Observer Pattern: Objects notify observers of state changes. Loose coupling. Used in event-driven systems.

Factory Pattern: Centralized object creation. Abstracts instantiation. Simplifies code and enables polymorphism.

Singleton Pattern: Single instance accessed globally. Used for databases, loggers. Thread-safety considerations for concurrent access.

Strategy Pattern: Encapsulates algorithms. Enables runtime switching. Used for different sorting/caching strategies.

10. CAPACITY PLANNING

Traffic Estimation: Forecast user growth and request patterns. Historical data and trends. Plan infrastructure proactively.

Resource Allocation: Balance cost and performance. Cloud auto-scaling handles dynamic loads. Reserve capacity for peak usage.

Cost Optimization: Monitor spending continuously. Reserved instances vs on-demand. Spot instances for non-critical workloads. Architectural efficiency.

Performance Testing: Load testing, stress testing, spike testing. Identify bottlenecks. Ensure system meets requirements under peak load.